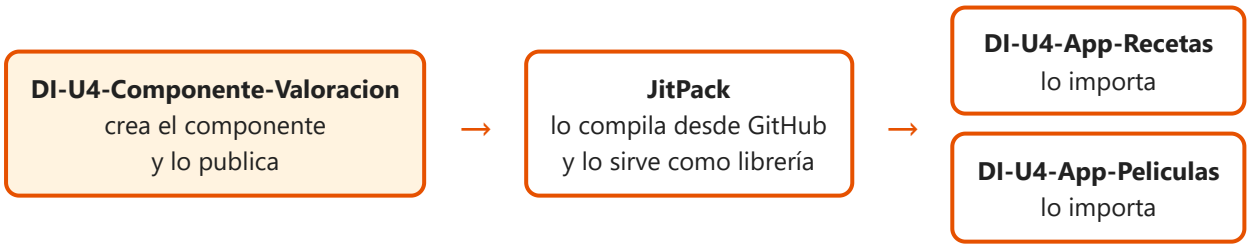
Crear un componente visual propio y reutilizarlo en otros proyectos

Guía paso a paso · Jetpack Compose · librería Android publicada en GitHub con JitPack

Módulo	Desarrollo de Interfaces (0488) · 2º DAM
Unidad	U4.1 Creación de componentes visuales (RA 3, CE a–g)
Componente	`Valoracion`: valoración con estrellas
Proyectos	github.com/IsmaelAndroid/**DI-U4-Componente-Valoracion** github.com/IsmaelAndroid/**DI-U4-App-Recetas** github.com/IsmaelAndroid/**DI-U4-App-Peliculas**
Profesor	Ismael Velasco · IES Rafael Alberti



Índice

1. **Qué vamos a construir** y por qué un componente se empaqueta como librería
2. **Crear el componente** (proyecto DI-U4-Componente-Valoracion)
    1. Crear el proyecto y el módulo de librería
    2. Configurar Gradle del módulo
    3. Escribir el componente: propiedades, evento y dibujo
    4. Previsualizarlo y usarlo en la app de demostración
    5. Probarlo
3. **Empaquetar y publicar** el componente: Maven, GitHub, versión y JitPack
4. **Importarlo en el proyecto 1**: App Recetas
5. **Importarlo en el proyecto 2**: App Películas
6. **Publicar una versión nueva** del componente
7. **Errores típicos** y cómo resolverlos

> **Requisitos.** Android Studio actualizado (AGP 9.1, Gradle 9.5), SDK de Android 15 (API 35), una cuenta de GitHub y Git configurado. Los tres proyectos de esta guía están terminados en GitHub: puedes descargarlos y comparar con lo tuyo en cualquier paso.

1. Qué vamos a construir

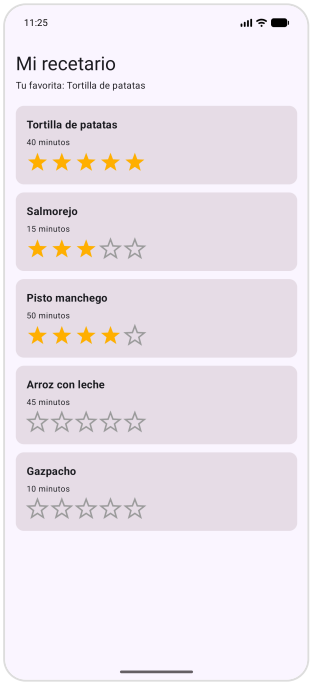
En la teoría hiciste `BadgeEstado` y `SelectorMes` dentro del mismo proyecto que los usaba. Ahora vamos un paso más allá: el componente vive en **su propio repositorio** y otros proyectos lo **importan** como importan Material 3, sin copiar ni un fichero.

Idea de la teoría	Cómo aparece en esta guía
Componente = propiedades + eventos	Parámetros de `Valoracion` y el evento `onValorCambiado`
Dibujar sobre la interfaz (CE b)	Cada estrella se pinta con `Canvas` y un `Path`
Pruebas del componente (CE e)	`ValoracionTest` con `createComposeRule`
Documentación (CE f)	KDoc del componente y `@Preview`
Empaquetado (CE g): el `.jar` → `.aar`	Módulo `com.android.library` + `maven-publish`
"Añadir a la paleta"	Una línea `implementation(...)` en el proyecto que lo usa

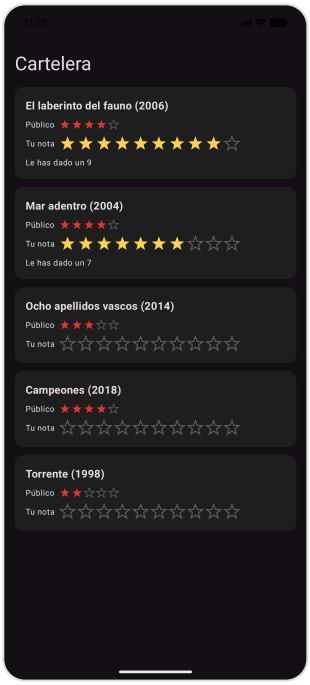
El componente es una **valoración con estrellas**: muestra de 0 a N estrellas encendidas y, si se le pasa un evento, deja que el usuario pulse para cambiar la nota.



App de demostración
(catálogo del componente)



Proyecto 1: Recetas
(lo importa)



Proyecto 2: Películas
(lo importa)

2. Crear el componente

2.1 Crear el proyecto y el módulo de librería

1. **File ▸ New ▸ New Project... ▸ Empty Activity**. Nombre `ComponenteValoracion`, paquete `com.dam.valoracion.demo`, lenguaje Kotlin, *Minimum SDK* API 24, *Build configuration language* Kotlin DSL.
2. Este proyecto ya tiene el módulo `:app`. Será la **app de demostración**: el sitio donde ves el componente funcionando, como la paleta del IDE.
3. Crea el módulo del componente: **File ▸ New ▸ New Module... ▸ Android Library**. *Module name* `valoracion`, *Package name* `com.dam.valoracion`, lenguaje Kotlin. **Finish**.
4. Comprueba que `settings.gradle.kts` incluye los dos módulos:

settings.gradle.kts (final del fichero)

```
rootProject.name = "ComponenteValoracion"
include(":valoracion")   // el componente: módulo de librería (produce un .aar)
include(":app")          // la app de demostración que enseña el componente
```

> **¿Por qué dos módulos?** Un módulo **application** genera un `.apk` que se instala. Un módulo **library** genera un `.aar` (*Android Archive*): el `.jar` de Java más recursos y manifiesto. El `.aar` no se instala: se **importa** desde otros proyectos.

2.2 Configurar Gradle del módulo

El asistente crea el módulo sin Compose. Hay que añadirlo en tres sitios.

a) En el catálogo de versiones, el plugin de librería y las librerías de pruebas:

gradle/libs.versions.toml (lo nuevo)

```
[versions]
junitAndroidx = "1.3.0"
espressoCore = "3.7.0"

[libraries]
androidx-ui-test-junit4 = { group = "androidx.compose.ui", name = "ui-test-junit4" }
androidx-ui-test-manifest = { group = "androidx.compose.ui", name = "ui-test-manifest" }
androidx-junit = { group = "androidx.test.ext", name = "junit", version.ref = "junitAndroidx" }
androidx-espresso-core = { group = "androidx.test.espresso", name = "espresso-core", version.ref =
"espressoCore" }

[plugins]
android-library = { id = "com.android.library", version.ref = "agp" }
```

b) En el `build.gradle.kts` raíz, se declara el plugin (sin aplicarlo):

build.gradle.kts (raíz)

```
plugins {
    alias(libs.plugins.android.application) apply false
    alias(libs.plugins.android.library) apply false
    alias(libs.plugins.kotlin.compose) apply false
}
```

c) El `build.gradle.kts` del módulo `valoracion` queda así:

valoracion/build.gradle.kts

```
plugins {
    alias(libs.plugins.android.library)   // librería, no aplicación
    alias(libs.plugins.kotlin.compose)
    `maven-publish`                        // para publicarlo (apartado 3)
}

android {
    namespace = "com.dam.valoracion"
    compileSdk = 35
    defaultConfig {
        minSdk = 24
        testInstrumentationRunner = "androidx.test.runner.AndroidJUnitRunner"
    }
    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_11
        targetCompatibility = JavaVersion.VERSION_11
    }
    buildFeatures { compose = true }
}

dependencies {
    // api y no implementation: quien use el componente también necesita ver Compose
    api(platform(libs.androidx.compose.bom))
    api(libs.androidx.ui)
    api(libs.androidx.ui.graphics)
    api(libs.androidx.material3)
    implementation(libs.androidx.ui.tooling.preview)
    debugImplementation(libs.androidx.ui.tooling)

    androidTestImplementation(platform(libs.androidx.compose.bom))
    androidTestImplementation(libs.androidx.ui.test.junit4)
    androidTestImplementation(libs.androidx.junit)
    androidTestImplementation(libs.androidx.espresso.core)   // reciente: ver apartado 7
    debugImplementation(libs.androidx.ui.test.manifest)
}
```

> **`api` o `implementation`.** `implementation` esconde la dependencia: solo la ve el módulo que la declara. `api` la **pasa** a quien te importa. El componente recibe un `Modifier` y un `Color`, que son tipos de Compose: quien lo use tiene que ver Compose, así que va con `api`.

Por último, borra del módulo lo que el asistente crea y no se usa: las carpetas `res/` vacías, `consumer-rules.pro` y `proguard-rules.pro`. El manifiesto de una librería se queda vacío, porque las actividades y el icono los pone la app:

valoracion/src/main/AndroidManifest.xml

```
<manifest />
```

Pulsa **Sync Now**.

2.3 Escribir el componente

Crea `valoracion/src/main/java/com/dam/valoracion/Valoracion.kt`. Vamos por partes.

Las propiedades. Cada parámetro es una propiedad. Solo `valor` es obligatoria; las demás tienen valor por defecto, así que el uso más simple es `Valoracion(valor = 3)`.

Valoracion.kt · la firma

```
/**
 * Componente: valoración con estrellas.
 * @param valor número de estrellas encendidas (se ajusta solo entre 0 y [maximo])
 * @param modifier posición y márgenes decididos por la pantalla que lo usa
 * @param maximo número total de estrellas
 * @param tamano lado de cada estrella
 * @param colorActivo color de las estrellas encendidas
 * @param colorInactivo color del contorno de las estrellas apagadas
 * @param onValorCambiado evento: se dispara con la estrella pulsada (1..maximo).
 *   Si es `null`, el componente es de solo lectura y no responde a pulsaciones.
 */
@Composable
fun Valoracion(
    valor: Int,
    modifier: Modifier = Modifier,
    maximo: Int = 5,
    tamano: Dp = 32.dp,
    colorActivo: Color = Color(0xFFFFB300),
    colorInactivo: Color = Color(0xFF9E9E9E),
    onValorCambiado: ((Int) -> Unit)? = null,
)
```

El cuerpo y el evento. Una fila con `maximo` estrellas. Si hay evento, cada estrella es pulsable y **avisa** con su posición. El componente **no guarda** la nota: eso lo hace quien lo usa.

Valoracion.kt · el cuerpo

```
{
    require(maximo > 0) { "maximo debe ser mayor que 0" }
    val valorSeguro = valor.coerceIn(0, maximo)

    Row(
        modifier = modifier
            .testTag("valoracion")
            // Lo que lee TalkBack: el número, no "cinco imágenes"
            .semantics { contentDescription = "$valorSeguro de $maximo estrellas" },
        horizontalArrangement = Arrangement.spacedBy(tamano / 8),
    ) {
        for (posicion in 1..maximo) {
            // Solo es pulsable si alguien escucha el evento
            val pulsable = if (onValorCambiado != null) {
                Modifier.clickable(onClickLabel = "Puntuar con $posicion") {
onValorCambiado(posicion) }
            } else {
                Modifier
            }
            Estrella(
                encendida = posicion <= valorSeguro,
                colorActivo = colorActivo,
                colorInactivo = colorInactivo,
                modifier = pulsable.testTag("estrella_$posicion").size(tamano),
            )
        }
    }
}
```

El dibujo. No hay ningún icono de estrella en Material 3 base, así que la dibujamos sobre un lienzo (`Canvas`, el `Graphics` de Swing). Una estrella de 5 puntas son 10 vértices que alternan un radio grande (las puntas) y uno pequeño (los huecos):

Valoracion.kt · el dibujo

```
@Composable
private fun Estrella(encendida: Boolean, colorActivo: Color, colorInactivo: Color,
                     modifier: Modifier = Modifier) {
    Canvas(modifier) {
        val lado = size.minDimension
        val forma = trazadoEstrella(lado)
        if (encendida) {
            drawPath(forma, colorActivo)                                        // rellena
        } else {
            drawPath(forma, colorInactivo, style = Stroke(width = lado * 0.07f)) // solo el borde
        }
    }
}

internal fun trazadoEstrella(lado: Float): Path {
    val centro = lado / 2f
    val radioExterior = lado / 2f * 0.92f      // margen para que el borde no se corte
    val radioInterior = radioExterior * 0.42f
    return Path().apply {
        for (i in 0 until 10) {
            val radio = if (i % 2 == 0) radioExterior else radioInterior
            val angulo = PI / 5 * i - PI / 2       // empieza en la punta de arriba
            val x = centro + (radio * cos(angulo)).toFloat()
            val y = centro + (radio * sin(angulo)).toFloat()
            if (i == 0) moveTo(x, y) else lineTo(x, y)
        }
        close()
    }
}
```

Visibilidad. `Valoracion` es **pública**: es lo que exporta la librería. `Estrella` es `private` y `trazadoEstrella` es `internal` (visible solo dentro del módulo): quien importe la librería no las ve ni depende de ellas, así que puedes cambiarlas sin romper a nadie.

2.4 Previsualizarlo y usarlo en la app de demostración

Al final de `Valoracion.kt` añade dos `@Preview`, uno por cada forma de uso. Se ven en la vista **Split** del editor sin arrancar el emulador.

Valoracion.kt · previsualizaciones

```
@Preview(name = "Valoracion · solo lectura", showBackground = true)
@Composable
private fun VistaPreviaSoloLectura() {
    Column(verticalArrangement = Arrangement.spacedBy(8.dp)) {
        Valoracion(valor = 0)
        Valoracion(valor = 3)
        Valoracion(valor = 7, maximo = 10, tamano = 20.dp, colorActivo = Color(0xFFE53935))
    }
}

@Preview(name = "Valoracion · interactiva", showBackground = true)
@Composable
private fun VistaPreviaInteractiva() {
    var valor by remember { mutableIntStateOf(2) }
    Valoracion(valor = valor, onValorCambiado = { valor = it })
}
```

Para usarlo desde `:app`, se añade el módulo como dependencia. Es la forma de importar un componente **dentro del mismo proyecto**:

app/build.gradle.kts

```
dependencies {
    implementation(project(":valoracion"))
    implementation(libs.androidx.core.ktx)
    implementation(libs.androidx.activity.compose)
    debugImplementation(libs.androidx.ui.tooling)
}
```

En `MainActivity.kt` de la app se importa con `import com.dam.valoracion.Valoracion` y se monta un catálogo con una tarjeta por cada forma de uso. El caso interactivo es el importante: **la pantalla guarda el estado** y el componente lo cambia a través del evento.

app/.../MainActivity.kt · fragmento

```
var nota by rememberSaveable { mutableIntStateOf(3) }
Valoracion(valor = nota, onValorCambiado = { nota = it })
Text("Has elegido $nota de 5")

Valoracion(valor = 4)                                    // solo lectura
Valoracion(valor = 7, maximo = 10, tamano = 20.dp,
           colorActivo = Color(0xFFE53935))              // propiedades cambiadas
```

Ejecuta **app**: verás el catálogo de la primera captura del apartado 1.

2.5 Probarlo

Crea `valoracion/src/androidTest/java/com/dam/valoracion/ValoracionTest.kt`. Las pruebas buscan los nodos por su `testTag` o su descripción, igual que haría TalkBack:

ValoracionTest.kt · dos de las cuatro pruebas

```
class ValoracionTest {
    @get:Rule
    val regla = createComposeRule()

    @Test
    fun sinEventoEsDeSoloLectura() {
        regla.setContent { Valoracion(valor = 2) }
        regla.onNodeWithTag("estrella_4").assertHasNoClickAction()
    }

    @Test
    fun alPulsarAvisaConLaEstrellaPulsada() {
        regla.setContent {
            var valor by remember { mutableIntStateOf(1) }
            Valoracion(valor = valor, onValorCambiado = { valor = it })
        }
        regla.onNodeWithTag("estrella_4").assertHasClickAction().performClick()
        regla.onNodeWithContentDescription("4 de 5 estrellas").assertExists()
    }
}
```

Con el emulador arrancado: clic derecho sobre la clase ▸ **Run 'ValoracionTest'**, o en la terminal `./gradlew :valoracion:connectedAndroidTest`.

3. Empaquetar y publicar el componente

Hasta aquí el componente solo se puede usar dentro de su proyecto. Para que **otro proyecto** lo importe hay que publicarlo en un repositorio Maven (el "almacén" de donde Gradle descarga las librerías). Usaremos **JitPack**: un repositorio Maven gratuito que compila la librería directamente desde GitHub.

3.1 Las coordenadas Maven

Toda librería se pide por tres datos: `grupo:artefacto:versión`. Se declaran al final de `valoracion/build.gradle.kts`:

valoracion/build.gradle.kts (añadir)

```
android {
    // ...lo que ya había...
    // Variante que se publica: la release, con el código fuente para leerlo desde el IDE
    publishing {
        singleVariant("release") { withSourcesJar() }
    }
}

afterEvaluate {
    publishing {
        publications {
            create<MavenPublication>("release") {
                groupId = "com.github.IsmaelAndroid"           // com.github.TU_USUARIO
                artifactId = "DI-U4-Componente-Valoracion"     // igual que el repositorio
                version = "1.0.0"                              // mayor.menor.parche
                from(components["release"])                    // el .aar ya construido
            }
        }
    }
}
```

> Con JitPack el grupo es siempre `com.github.` + tu usuario de GitHub, y la versión es el nombre de la etiqueta (tag) de Git. Poner aquí los mismos valores hace que la librería se pida igual en local que desde JitPack.

Comprueba en local que se empaqueta. En la terminal de Android Studio:

Terminal

```
./gradlew :valoracion:publishToMavenLocal
```

Se crea la carpeta `C:\Users\TU_USUARIO\.m2\repository\com\github\IsmaelAndroid\DI-U4-Componente-Valoracion\1.0.0\` con:

Fichero	Qué es
`…-1.0.0.aar`	El componente compilado (clases + recursos + manifiesto)
`…-1.0.0.pom` y `.module`	La "ficha" de la librería: coordenadas y dependencias (Compose, con `api`)
`…-1.0.0-sources.jar`	El código fuente, para que el IDE lo enseñe con Ctrl+clic

3.2 Decirle a JitPack cómo compilarlo

Crea `jitpack.yml` en la raíz del proyecto. Así JitPack usa Java 17 y solo publica el módulo del componente (la app de demostración no le interesa a nadie):

jitpack.yml

```
jdk:
  - openjdk17
install:
  - ./gradlew :valoracion:publishToMavenLocal
```

3.3 Subirlo a GitHub (repositorio público)

Desde la terminal, en la carpeta del proyecto (o con **VCS ▸ Share Project on GitHub**):

Terminal

```
git init -b main
git add .
git update-index --chmod=+x gradlew        # JitPack compila en Linux: gradlew debe ser ejecutable
git commit -m "Componente Valoracion"
git remote add origin https://github.com/IsmaelAndroid/DI-U4-Componente-Valoracion.git
git push -u origin main
```

> El repositorio tiene que ser **público**. JitPack gratuito solo lee repositorios públicos; con uno privado, la importación falla con *Failed to resolve*.

3.4 Crear la versión 1.0.0

Una versión es una **etiqueta** de Git. Puedes crearla en GitHub (**Releases ▸ Draft a new release ▸ Choose a tag:** `1.0.0` **▸ Publish release**) o desde la terminal:

Terminal

```
git tag 1.0.0
git push origin 1.0.0
```

3.5 Comprobarlo en JitPack

1. Entra en **jitpack.io**, escribe `IsmaelAndroid/DI-U4-Componente-Valoracion` y pulsa **Look up**.
2. Aparece la versión `1.0.0`. Pulsa **Get it**: JitPack compila la librería (la primera vez tarda 2-3 minutos).
3. Cuando el icono del **Log** se ponga verde, la librería está lista. Si sale rojo, ábrelo: es el mismo error que verías en Android Studio (apartado 7).

JitPack también te enseña cómo importarla. Es lo que vamos a hacer en los dos apartados siguientes.

4. Importarlo en el proyecto 1: App Recetas

Un recetario en el que cada receta se valora con estrellas. Usa el componente **tal cual**, con sus valores por defecto. Para importar una librería de GitHub hacen falta **tres cambios**.

4.1 Crear el proyecto

File ▸ **New** ▸ **New Project...** ▸ **Empty Activity**. Nombre `AppRecetas`, paquete `com.dam.recetas`, API 24, Kotlin DSL. Es un proyecto **distinto** y en otra carpeta: no tiene el código del componente.

4.2 Cambio 1: añadir el repositorio de JitPack

Gradle solo busca librerías en los repositorios que le digas. Por defecto son `google()` y `mavenCentral()`; JitPack hay que añadirlo:

settings.gradle.kts

```
dependencyResolutionManagement {
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
    repositories {
        google()
        mavenCentral()
        maven("https://jitpack.io")       // compila el componente desde GitHub y nos lo sirve
    }
}
```

> Va en `settings.gradle.kts`, no en `build.gradle.kts`. Con `FAIL_ON_PROJECT_REPOS`, un repositorio declarado en un `build.gradle.kts` hace fallar la sincronización.

4.3 Cambio 2: declarar la librería en el catálogo

gradle/libs.versions.toml

```
[versions]
# ...
valoracion = "1.0.0"

[libraries]
# ...
valoracion = { group = "com.github.IsmaelAndroid", name = "DI-U4-Componente-Valoracion", version.ref
= "valoracion" }
```

4.4 Cambio 3: añadir la dependencia

app/build.gradle.kts

```
dependencies {
    implementation(libs.valoracion)      // el componente, desde GitHub vía JitPack
    implementation(libs.androidx.core.ktx)
    // ...
}
```

Sin catálogo de versiones, los cambios 2 y 3 se resumen en una línea: `implementation("com.github.IsmaelAndroid:DI-U4-Componente-Valoracion:1.0.0")`.

Pulsa **Sync Now**. La primera vez Gradle descarga el `.aar`. Para comprobarlo, abre la vista **Project** ▸ **External Libraries**: aparece `com.github.IsmaelAndroid:DI-U4-Componente-Valoracion:1.0.0`.

4.5 Usar el componente

Se importa como cualquier otra clase. El estado (la lista de recetas con sus notas) vive en la pantalla; el evento del componente lo actualiza:

app/src/main/java/com/dam/recetas/MainActivity.kt · fragmentos

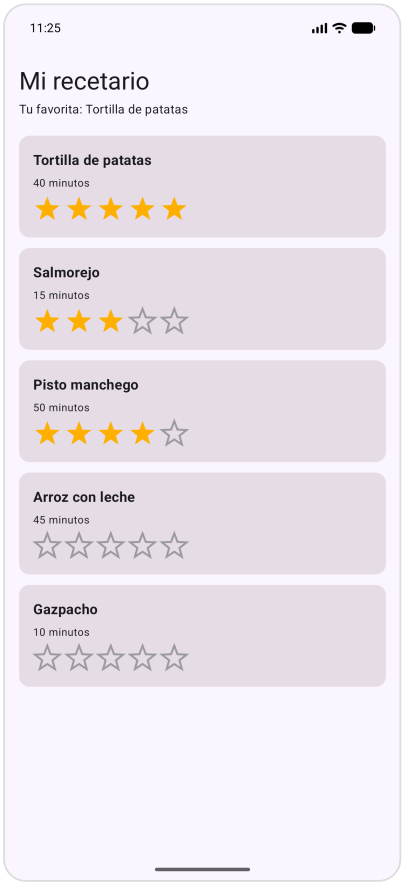
```
import com.dam.valoracion.Valoracion   // viene de la librería, no de este proyecto

data class Receta(val nombre: String, val minutos: Int, val nota: Int = 0)

val recetas = remember {
    mutableStateListOf(Receta("Tortilla de patatas", 40), Receta("Salmorejo", 15), /* ... */)
}

items(recetas.indices.toList(), key = { recetas[it].nombre }) { i ->
    TarjetaReceta(
        receta = recetas[i],
        onNota = { nueva -> recetas[i] = recetas[i].copy(nota = nueva) },
    )
}

@Composable
private fun TarjetaReceta(receta: Receta, onNota: (Int) -> Unit) {
    Card(Modifier.fillMaxWidth()) {
        Column(Modifier.padding(16.dp)) {
            Text(receta.nombre, style = MaterialTheme.typography.titleMedium)
            Text("${receta.minutos} minutos")
            Valoracion(valor = receta.nota, onValorCambiado = onNota)
        }
    }
}
```



App Recetas tras valorar tres recetas

5. Importarlo en el proyecto 2: App Películas

Una cartelera. Repite **exactamente** los cambios 1, 2 y 3 del apartado 4 en un proyecto nuevo (`AppPeliculas`, paquete `com.dam.peliculas`). La diferencia está en el uso: el **mismo componente** aparece dos veces en cada tarjeta, configurado de forma distinta solo con sus propiedades.

Uso	Propiedades que cambia	Resultado
Media del público	sin `onValorCambiado`, `tamano = 16.dp`, `colorActivo` rojo	pequeña y de solo lectura
Tu nota	`maximo = 10`, `tamano = 24.dp`, `colorActivo` amarillo, con evento	sobre 10 e interactiva

app/src/main/java/com/dam/peliculas/MainActivity.kt · la tarjeta

```
import com.dam.valoracion.Valoracion

data class Pelicula(val titulo: String, val anio: Int, val mediaPublico: Int, val tuNota: Int = 0)

// Uso 1: solo lectura (sin onValorCambiado), pequeño y en rojo
Valoracion(
    valor = pelicula.mediaPublico,
    tamano = 16.dp,
    colorActivo = Color(0xFFE53935),
    colorInactivo = Color(0xFF616161),
)

// Uso 2: interactiva y sobre 10
Valoracion(
    valor = pelicula.tuNota,
    maximo = 10,
    tamano = 24.dp,
    colorActivo = Color(0xFFFFD54F),
    colorInactivo = Color(0xFF616161),
    onValorCambiado = onNota,
)
```



App Películas: el componente en dos configuraciones

Lo que has conseguido. Dos proyectos distintos usan el mismo componente sin tener su código. Si mañana el componente mejora, no se toca ninguno de los dos: basta con subir la versión (apartado 6).

6. Publicar una versión nueva del componente

Supón que quieres que el componente permita borrar la nota pulsando la misma estrella otra vez.

1. En **DI-U4-Componente-Valoracion**, cambia el código y sube la versión en `valoracion/build.gradle.kts`: `version = "1.1.0"`. Cambio compatible = sube el número del medio; arreglo = el último; cambio que rompe a quien lo usa (quitar o renombrar un parámetro) = el primero.
2. Prueba en la app de demostración, haz commit y sube: `git push`.
3. Crea la etiqueta: `git tag 1.1.0` y `git push origin 1.1.0`.
4. En cada app que lo use, cambia **una línea** del catálogo: `valoracion = "1.1.0"`, y sincroniza.

Las apps que no cambien la línea siguen con la 1.0.0: cada proyecto decide cuándo actualizar.

Alternativa sin GitHub: Maven Local

Para probar en tu ordenador antes de publicar (es la opción que aparece en la teoría): en la librería `./gradlew :valoracion:publishToMavenLocal`, y en la app, en `settings.gradle.kts`, añade `mavenLocal()` a los repositorios. La dependencia se escribe igual. Solo funciona en **tu** ordenador: un compañero que descargue la app no tendrá la librería.

7. Errores típicos

Error	Causa	Solución
Failed to resolve: com.github.IsmaelAndroid:DI-U4-Componente-Valoracion:1.0.0	Falta el repositorio de JitPack, el repositorio de GitHub es privado o no existe la etiqueta	Revisa `maven("https://jitpack.io")` en `settings.gradle.kts`, que el repositorio sea público y que exista el tag `1.0.0`
Build was configured to prefer settings repositories over project repositories	JitPack declarado en un `build.gradle.kts`	Muévelo a `dependencyResolutionManagement` de `settings.gradle.kts`
Unresolved reference: Valoracion	Falta el `import` o la dependencia	`import com.dam.valoracion.Valoracion` y `implementation(libs.valoracion)`
La app no encuentra clases de Compose al usar el componente	Compose declarado con `implementation` en la librería	En la librería, Compose con `api` (apartado 2.2)
JitPack: log en rojo con *Permission denied: ./gradlew*	`gradlew` subido sin permiso de ejecución	`git update-index --chmod=+x gradlew`, commit, y una etiqueta nueva
JitPack: *Android Gradle plugin requires Java 17*	JitPack compila con un Java antiguo	`jitpack.yml` con `jdk: - openjdk17`
He cambiado el componente y la app sigue igual	La app sigue pidiendo la versión antigua (o Gradle la tiene en caché)	Nueva etiqueta y nueva versión en el catálogo. Nunca reutilices una etiqueta ya publicada
Todas las pruebas fallan con *NoSuchMethodException: InputManager.getInstance*	Espresso antiguo (el que trae el BOM de Compose) en un emulador con Android reciente	Añade `espresso-core` 3.7.0 en `androidTestImplementation`
Toda la estrella se dibuja pero la punta sale cortada	El borde del `Stroke` se pinta por fuera del lienzo	Deja margen en el radio (`* 0.92f`)

Resumen

Paso	Dónde	Qué se hace
Crear	Librería	Módulo `com.android.library`, Compose con `api`, componente con propiedades + evento
Probar	Librería	`@Preview`, app de demostración con `project(":valoracion")`, `ValoracionTest`
Empaquetar	Librería	`maven-publish` con `grupo:artefacto:versión` y `jitpack.yml`
Publicar	GitHub	Repositorio público + etiqueta `1.0.0` → JitPack lo compila
Importar	Cada app	Repositorio JitPack + entrada en el catálogo + `implementation(...)` + `import`